

Dependency Inversion Principle C++



explain dependency inversion principle and give example in c++

Sure! The **Dependency Inversion Principle (DIP)** is one of the **SOLID** principles of object-oriented design. Here's a breakdown and a C++ example to help you understand it clearly.

Dependency Inversion Principle (DIP)

The principle states:

1. **High-level modules should not depend on low-level modules.** Both should depend on abstractions.
2. **Abstractions should not depend on details.** Details (concrete implementations) should depend on abstractions.

In simpler terms: **code should depend on interfaces or abstract classes, not on concrete classes.** This makes your system more flexible and easier to maintain or extend.

Without DIP (Bad Example)

```
cpp

#include <iostream>

class LightBulb {
public:
    void turnOn() {
        std::cout << "LightBulb: ON\n";
    }

    void turnOff() {
        std::cout << "LightBulb: OFF\n";
    }
};

class Switch {
    LightBulb& bulb;
public:
    Switch(LightBulb& b) : bulb(b) {}

    void operate() {
        bulb.turnOn();
    }
};

int main() {
    LightBulb bulb;
    Switch s(bulb);
    s.operate();
    return 0;
}
```

❌ Problem:

- `Switch` depends directly on the concrete `LightBulb` class.
- We can't easily switch to a `LEDLight` or `SmartLight` without modifying `Switch`.

✅ With DIP (Good Example)

```
cpp
#include <iostream>

// Abstraction
class IDevice {
public:
    virtual void turnOn() = 0;
    virtual void turnOff() = 0;
    virtual ~IDevice() = default;
};

// Concrete implementation
class LightBulb : public IDevice {
public:
    void turnOn() override {
        std::cout << "LightBulb: ON\n";
    }
    void turnOff() override {
        std::cout << "LightBulb: OFF\n";
    }
};

// High-level module depends on abstraction
class Switch {
    IDevice& device;
public:
    Switch(IDevice& d) : device(d) {}

    void operate() {
        device.turnOn();
    }
};

int main() {
    LightBulb bulb;
    Switch s(bulb);
    s.operate();
    return 0;
}
```

✅ Benefits:

- `Switch` now depends on `IDevice`, an abstraction.
- We can plug in any device (e.g., `Fan`, `Heater`, etc.) that implements `IDevice` without changing `Switch`.

Want me to show how to extend this with multiple devices or use smart pointers for better memory management?



High-level modules should not depend on low-level modules.
what exactly has high level module

Great question! Understanding what a **high-level module** is makes the Dependency Inversion Principle (DIP) much clearer.

What is a High-Level Module?

A **high-level module** is **responsible for complex logic or coordination** of tasks in your program. It **controls behavior**, makes decisions, or orchestrates other parts of the system.

Think of it like the **brains** or **managers** of your application — it defines *what* needs to happen, but not *how* it should happen.

What is a Low-Level Module?

A **low-level module** is more like a **worker**. It performs specific, detailed tasks — like reading from a database, writing to a file, turning on a light, or sending a message.

Analogy

Imagine a company:

- The **CEO** or **Manager** (high-level module) says "We need a report on sales."
- The **Employee** (low-level module) actually goes and creates the report.

Now, imagine if the CEO directly told the employee *how* to do every step. That would make it hard to change anything (like using a new tool for reporting). Instead, it's better if both agree on a **standard report format (abstraction)** — then anyone who knows that format can provide the report.

In Code Terms

In our earlier example:

- ``Switch`` is the **high-level module** — it's controlling the behavior (deciding when to turn the device on/off).
- ``LightBulb`` is the **low-level module** — it knows how to turn on/off a light.
- ``IDevice`` is the **abstraction/interface** between them.

By depending on ``IDevice``, ``Switch`` can work with any device, not just ``LightBulb``.

Key Insight

High-level modules = modules that contain business logic or orchestration.

Low-level modules = modules that deal with system-level details or concrete implementations.

Would you like me to walk through another real-world example (like a payment processor, logging system, etc.)?



sure

Awesome! Let's walk through a **real-world example** using the **payment processing system**, which is super common in applications like e-commerce, subscriptions, or fintech apps.

Scenario: Payment Processing System

Goal:

You're building an app that charges users. You want to be able to swap out different payment providers (like Stripe, PayPal, Square, etc.) **without rewriting your business logic**.

Without DIP (Bad Design)

```
cpp

#include <iostream>

class StripePayment {
public:
    void makePayment(double amount) {
        std::cout << "Processing $" << amount << " with Stripe.\n";
    }
};

class PaymentProcessor {
    StripePayment stripe;
public:
    void process(double amount) {
        // Direct dependency on a specific provider
        stripe.makePayment(amount);
    }
};

int main() {
    PaymentProcessor processor;
    processor.process(99.99);
    return 0;
}
```

Problems:

- `PaymentProcessor` is a high-level module that contains business logic (when and how to process payments).
- But it depends **directly** on `StripePayment`, a low-level detail.
- If you want to switch to PayPal? You need to **modify** `PaymentProcessor`, which breaks **Open/Closed Principle** too.

✓ With DIP (Good Design)

```
cpp

#include <iostream>
#include <memory>

// Abstraction
class IPaymentService {
public:
    virtual void pay(double amount) = 0;
    virtual ~IPaymentService() = default;
};

// Concrete 1
class StripePayment : public IPaymentService {
public:
    void pay(double amount) override {
        std::cout << "Processing $" << amount << " with Stripe.\n";
    }
};

// Concrete 2
class PayPalPayment : public IPaymentService {
public:
    void pay(double amount) override {
        std::cout << "Processing $" << amount << " with PayPal.\n";
    }
};

// High-level module
class PaymentProcessor {
    std::shared_ptr<IPaymentService> service;
public:
    PaymentProcessor(std::shared_ptr<IPaymentService> svc) : service(svc) {}

    void process(double amount) {
        // Only depends on abstraction
        service->pay(amount);
    }
};

int main() {
    std::shared_ptr<IPaymentService> service = std::make_shared<PayPalPayment>();
    PaymentProcessor processor(service);
    processor.process(49.99);
    return 0;
}
```

✓ Benefits:

- `PaymentProcessor` (high-level) depends on `IPaymentService`, not on `StripePayment` or `PayPalPayment`.
- You can swap implementations with **zero change** to business logic.
- This design is **open for extension, closed for modification**, and follows **DIP**.

Would you like to see how this could scale with dependency injection frameworks or templates?



why are you using shared pointer and not unique

🔥 Great catch! Let's dig into **why I used `std::shared_ptr` instead of `std::unique_ptr`**, and when you'd want to use one over the other.

🧠 First — What's the Difference?

Pointer Type	Ownership Model	Copyable?	Typical Use
<code>std::unique_ptr</code>	Sole ownership	❌ No	When one object <i>exclusively owns</i> the resource.
<code>std::shared_ptr</code>	Shared ownership	✅ Yes	When multiple parts of the program <i>share ownership</i> of a resource.

✅ When to Use `unique_ptr`

- If the **high-level module owns the dependency outright**.
- There is **no reason for anyone else to share or keep a copy** of the dependency.
- Clear ownership — when the processor dies, the payment service should too.

Example with `unique_ptr`:

```
cpp
class PaymentProcessor {
    std::unique_ptr<IPaymentService> service;
public:
    PaymentProcessor(std::unique_ptr<IPaymentService> svc)
        : service(std::move(svc)) {}

    void process(double amount) {
        service->pay(amount);
    }
};

int main() {
    auto service = std::make_unique<StripePayment>();
    PaymentProcessor processor(std::move(service));
    processor.process(29.99);
}
```

✅ **This is often preferred** — it's cheaper (no ref counting), safer (no accidental copies), and more explicit.

✅ When to Use `shared_ptr`

Use `shared_ptr` if:

- Multiple components need **shared access** to the same instance.
- You're **registering services** somewhere (like in a dependency injection container).
- The object's lifetime is **not clearly tied to a single owner**.

- You're prototyping or don't want to deal with ownership yet (common early on, but risky long-term).

Rule of Thumb

Default to ``unique_ptr``. Only use ``shared_ptr`` when you have a *good reason* for shared ownership.

Would you like me to show an example with raw pointers + lifetime management or a custom factory that uses ``unique_ptr``?



So why you used service as shared pointer in the above example? Since `PaymentProcessor` owns the service, shouldn't it be unique?

Exactly! You're absolutely right — and I appreciate you calling that out.

The Correct Design Choice

In the example, ``PaymentProcessor`` **fully owns** the payment service. No one else needs or should access that service instance. That makes ``std::unique_ptr`` the **better choice** — it's simpler, more efficient, and clearly expresses ownership.

Why I used ``shared_ptr`` earlier?

Mostly for **demo convenience** and flexibility when swapping implementations or passing ownership around in simple examples. But from a **design correctness standpoint**, using ``shared_ptr`` there was **unnecessary and less ideal**.

Here's the corrected version using ``unique_ptr``:

```
cpp

#include <iostream>
#include <memory>

class IPaymentService {
public:
    virtual void pay(double amount) = 0;
    virtual ~IPaymentService() = default;
};

class StripePayment : public IPaymentService {
public:
    void pay(double amount) override {
        std::cout << "Processing $" << amount << " with Stripe.\n";
    }
};

class PayPalPayment : public IPaymentService {
public:
    void pay(double amount) override {
        std::cout << "Processing $" << amount << " with PayPal.\n";
    }
};
```

```

    }
};

class PaymentProcessor {
    std::unique_ptr<IPaymentService> service;
public:
    PaymentProcessor(std::unique_ptr<IPaymentService> svc)
        : service(std::move(svc)) {}

    void process(double amount) {
        service->pay(amount);
    }
};

int main() {
    auto service = std::make_unique<StripePayment>();
    PaymentProcessor processor(std::move(service));
    processor.process(29.99);
}

```

💡 Key Takeaway:

- Use `unique_ptr` when the high-level module owns the dependency exclusively — like here.
- Use `shared_ptr` only if the service needs to be **shared**, like if you injected the same logger, config, or DB connection across many components.

Want me to show a version with **constructor injection and DI container** style design too?



can you give an example of logger service with shared pointers?

Absolutely! A **logger** is a perfect example where `std::shared_ptr` actually **makes sense** — because:

- It's often used by **many parts of the system**.
- You typically want to **share a single instance** (singleton-style) to avoid creating multiple log files or outputs.
- You don't want to tightly couple everything to a global object.

📦 Scenario:

Multiple components (e.g., `PaymentProcessor`, `UserService`, etc.) all want to log messages using the same `Logger` instance.

✅ Logger Service Using `std::shared_ptr`

```

cpp

#include <iostream>
#include <memory>
#include <string>

// Logger Interface

```

```
class ILogger {
public:
    virtual void log(const std::string& message) = 0;
    virtual ~ILogger() = default;
};

// Concrete Logger
class ConsoleLogger : public ILogger {
public:
    void log(const std::string& message) override {
        std::cout << "[Log]: " << message << "\n";
    }
};

// A sample service that uses the logger
class PaymentProcessor {
    std::shared_ptr<ILogger> logger;
public:
    PaymentProcessor(std::shared_ptr<ILogger> log) : logger(log) {}

    void process(double amount) {
        logger->log("Processing payment of $" + std::to_string(amount));
    }
};

// Another service that uses the same logger
class UserService {
    std::shared_ptr<ILogger> logger;
public:
    UserService(std::shared_ptr<ILogger> log) : logger(log) {}

    void createUser(const std::string& name) {
        logger->log("Creating user: " + name);
    }
};

int main() {
    auto sharedLogger = std::make_shared<ConsoleLogger>();

    PaymentProcessor paymentProcessor(sharedLogger);
    UserService userService(sharedLogger);

    paymentProcessor.process(49.99);
    userService.createUser("Alice");

    return 0;
}
```

✓ Output:

markdown

```
`[Log]: Processing payment of $49.990000
[Log]: Creating user: Alice`
```

🧠 Why shared_ptr is right here:

- `sharedLogger` is shared among multiple services.
- No single service "owns" the logger; it's **co-owned** by whoever needs it.
- The logger's lifetime is managed automatically.

Bonus Tip:

In real-world projects, loggers are often passed as shared singletons or registered in a **DI container**. This pattern aligns well with ``shared_ptr``.

Let me know if you'd like to see a file logger, timestamped logger, or logger factory too!